

PROJECT PROPOSAL

Web-Based Hospital Crowd Management & Test Verdict System

Project Title:	UpacharVerdict: Online Medical Test Interpretation Platform
Target Sector:	Healthcare Infrastructure & Outpatient Department (OPD) Optimization in Nepal
Proposed Stack:	Python, Django (MVT Architecture), SQLite/PostgreSQL, Tailwind CSS
Implementation:	Solo Dev Portfolio & Future Core Final Year Project (FYP)

1. Introduction & Background

In the context of Nepal's healthcare ecosystem, major public and private hospitals (such as Bir Hospital, Tribhuvan University Teaching Hospital, and Patan Hospital) encounter severe daily overcrowding. A substantial and critical portion of this crowd consists of patients who have already undergone laboratory or radiology testing and are physically returning to the facility solely to show their raw report sheets to a doctor for a definitive diagnostic verdict (popularly known as the *"Report Dekhaune"* crowd).

This recurring physical influx burdens the hospital infrastructure, leads to prolonged lines in Outpatient Departments (OPDs), and exhausts limited medical personnel. Furthermore, patients frequently travel from distant rural districts, incurring unnecessary financial expenses for transport, food, and hotel accommodation just to hear a 30-second text summary or verbal confirmation of their lab results.

2. Problem Statement

The current diagnostic pipeline relies heavily on physical interactions for post-test evaluations. Patients receive raw data outputs (numerical ranges, scans) which they cannot comprehend without a physician. Consequently, the hospital suffers from:

- **Infrastructural Strain:** Unnecessary physical presence increases congestion in corridors, waiting lounges, and billing counters.
- **Inappropriate Allocation of Medical Hours:** Doctors waste valuable active consultation hours explaining normal or routine test parameters instead of diagnosing new critical cases.
- **Geographic and Economic Vulnerability:** Rural Nepalese patients bear heavy financial and logistical challenges for an exchange that can securely take place online.

3. Proposed Solution

The proposed web application, **UpacharVerdict**, provides a secure, lightweight platform where laboratories upload digitized diagnostic reports, physicians attach a direct plain-text actionable medical interpretation (the "Verdict"), and patients pull up this information on demand. By converting the post-test consultation into an

asynchronous digital workflow, the physical presence requirement drops dramatically, resolving core overcrowding vectors.

4. Target Users & System Roles

The application segments user access strictly into three specific primary roles to mirror real-world hospital operational hierarchies:

1. Laboratory Technicians (Data Ingestion Tier)

Responsible for processing medical tests and launching records into the digital environment. They input key patient identifiers and register the baseline testing entry.

2. Medical Consultants / Doctors (Diagnostic Verdict Tier)

The core clinical users. They review the baseline data, append their definitive assessment/clinical decisions, and instantly authorize online publication.

3. Patients (End-Consumer Portal Tier)

The diagnostic consumers. They securely access their targeted record using unique identifiers, circumventing the need to physically queue inside the facility.

5. Full Feature Specification Matrix

System Module	Detailed Functional Capabilities
Lab Administration Panel	• Manual record creation with generation of unique Patient IDs. • Data fields for Patient Name, Phone Number, and Test Type (e.g., Thyroid, Blood Sugar). • Immediate persistence initialization (sets initial status flag to 'Pending').
Doctor Clinical Dashboard	• Centralized queue displaying all unreviewed records filtering out finalized reports. • Single-click detailed view interface to read the record profile. • Rich text verdict submission module to update system state. • Automatic state transition engine toggling status flags to 'Reviewed'.
Patient Request Portal	• Ultra-clean, high-accessibility mobile-responsive search dashboard landing page. • Dual-factor lookup verification matching unique Patient ID against verified Phone Numbers. • Dynamic conditional view: Displays standard processing notices if 'Pending' or full medical notes and doctor identification details if 'Reviewed'.
Notification Engine	• Automated notification simulation mechanism built directly into backend view functions. • Console-logged trigger logs modeling pipeline execution to handle SMS dispatch integrations.

6. Technical Architecture & Constraints

To ensure rapid deployment within a strict timeline, the development architecture bypasses the integration traps of separate frontend-backend layers, adopting a robust server-side architecture:

- **Backend Framework:** Django 5.x utilizing Python 3.x, driving secure, performant database mappings.
- **Database Management:** Built-in SQLite for localized prototyping, easily translating to Enterprise PostgreSQL (hosted via Neon.tech) for deployment.
- **Database Engine Layer (ORM):** Django Object-Relational Mapper completely handles database operations without writing raw SQL.
- **Frontend Rendering:** HTML5 paired with Tailwind CSS via a Content Delivery Network (CDN) to ensure lightning-fast UI design without heavy compilation dependencies.

7. Implementation Roadmap (30-Day Agile Timeline)

The execution strategy utilizes project-based learning to transition smoothly from database design to final production:

1. **Days 1–5 (Foundations & Setup):** Python data types mastery, virtual environment virtualization, Django core framework installation, and development server testing.
2. **Days 6–10 (Database Architecture & Administration):** Declaring object schemas within `models.py`, tracking migrations, and activating the customized Django Admin system.
3. **Days 11–20 (Controller Architecture & Queries):** Designing routing structures in `urls.py` and coding view functions to query data fields and drive conditional rendering workflows.
4. **Days 21–25 (User Experience & Tailwind Styling):** Building responsive base layouts, dashboards, input structures, and polish.
5. **Days 26–30 (Deployment & Portfolio Architecture):** Shifting to cloud hosting via Render, establishing remote production databases, drafting documentation, and publishing the active system URL.

8. Future Expansion Vector (Final Year Project Blueprint)

While the initial 30-day baseline version builds immediate internship competency, the architecture intentionally allows easy modular scaling for an advanced college Final Year Project (FYP):

- **Decoupled Architecture Integration:** Re-platforming the Django layer to serve as a pure Rest API using Django Rest Framework (DRF), and constructing a single-page frontend application via React.js.
- **Production OTP Authentication:** Swapping the standard login flow for a real-world One-Time Password (OTP) validation framework via SMS gateways like Aakash SMS or Sparrow SMS.
- **Integrated Media Storage:** Plugging in Cloudinary or Amazon S3 buckets to handle native PDF report hosting and DICOM medical imaging file viewers.